Two Architectures, One Trading System

`original` vs `phase-5-sizing-exits` — what each branch does, and why they differ

Repository: AI_Brain_Correlation_Hedging · Generated 2026-09-10 · MetaTrader 5 only

The short version

This repository contains two complete and genuinely different answers to the same question: *how do you let a language model trade an account without it eventually destroying that account?*

The `original` branch answers it by **learning from its own losses**. The `phase-5-sizing-exits` branch answers it by **refusing to trust the model at all**, and putting a deterministic risk engine between the model and the broker. Neither is wrong. They are optimised for different things, and the table below is the honest summary of what each buys you.

original

7 modules · ~2,400 lines · 0 tests

- **Self-learning.** An LLM auditor reads the losing trades and writes rules that penalise those setups next time.
- **Flat-file state** — `memory.json`, `new_rules.json`.
- **9 symbols**, 1.0% risk per trade.
- Sizing from stop distance and tick value.
- Duplicate and hedge guards in the loop.
- Runs 24/7 detached via WMI on Windows.
- One dashboard, including a Self-Learning Desk.

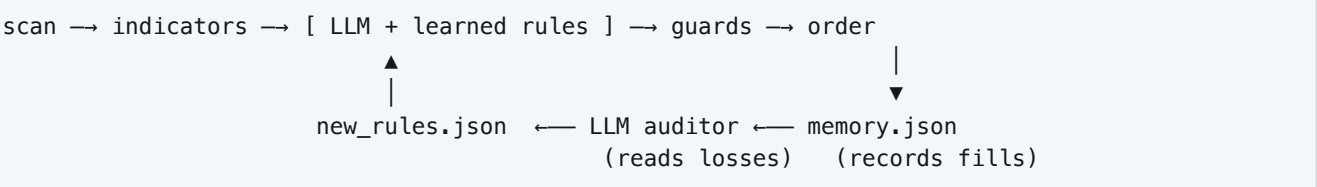
phase-5-sizing-exits

38 modules · ~16,700 lines · 255 tests

- **Deterministic risk engine** with 14 ordered checks holding sole authority over every order.
- **SQLite**, 13 tables, reconciled against MT5 in 4 passes.
- **4 symbols**, 0.25% risk, prop-firm rule profiles in YAML.
- Versioned strategies — every decision is reproducible.
- Crash-safe idempotent execution via comment tags.
- CI on Python 3.12 and 3.13.
- Built to answer one question: does the AI score predict anything?

What the `original` branch does

Every 30 seconds it walks nine symbols. For each one it pulls 250 H1 and 250 Daily bars, computes EMA(200), RSI(14), ATR(14) and relative volume on both frames, and asks DeepSeek for a decision. What makes it interesting is what happens next.



Every fill is written to `memory.json` with the full market context that existed at entry — not just the price, but ATR, relative volume, RSI and what all eight other instruments were quoting. After each scan the bot asks MT5 which positions have closed and stamps the realised P/L onto those records.

Once ten trades have closed, the auditor takes over. It hands the *losing* trades to DeepSeek acting as a Chief Risk Officer and asks a single narrow question: what do these have in common? What comes back is a set of rules — a symbol, a setup, a penalty — that are injected into every subsequent prompt as mandatory confidence deductions. A setup that lost four times arrives at the model pre-penalised and has to clear a higher bar than it did the first time.

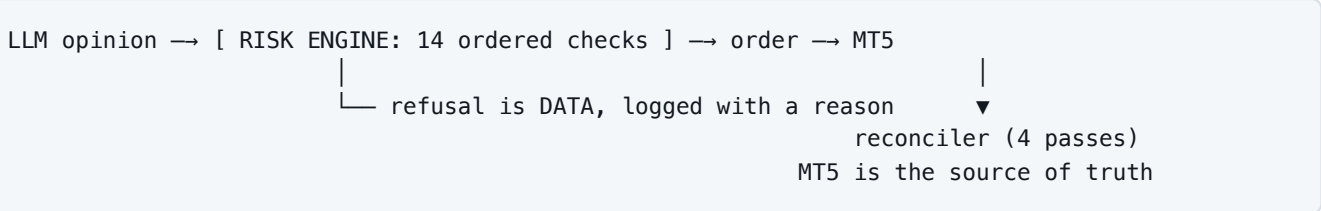
The three guardrails that matter

GUARDRAIL	WHAT IT PREVENTS
Penalty clamp 15–30	An unclamped model will happily propose a 90-point penalty, silently disabling a symbol forever on a sample of four trades.
Minimum 2 samples	One losing trade is an anecdote. Rules built on a single loss are superstition with a JSON schema.
Merge, never stack	Re-confirming a rule updates it in place. Without this, three audits agreeing would compound into a 60-point penalty.

> **The genuinely good idea here** is storing the full market context with each fill. Knowing a trade lost tells you nothing you can act on. Knowing it lost on 0.4× relative volume, for the fifth time, is a rule. Most retail bots record only price and outcome and can therefore never learn anything structural.

What the `phase-5-sizing-exits` branch does

The same core loop, rebuilt over five gated phases on the premise that **the model is an untrusted input**. The architecture is three layers that do not trust each other:



LAYER	GUARANTEE IT PROVIDES
AI brain	`get_ai_decision()` is contractually incapable of raising. Every failure returns a safe HOLD, so a malformed response can never reach execution.
Risk engine	Fourteen ordered checks — kill switch, score floor, spread, hedging ban, position cap, per-trade risk, daily loss, loss streak, account floor, net exposure. First failure wins and every check is logged, including the ones never reached.
Sizing & stops	Volume floored to the broker's step so realised risk is always at or below target. A minimum lot that exceeds the budget is refused, never silently traded.
Execution	Intent written to SQLite *before* the order is sent. A crash mid-flight is resolved by asking MT5, never by re-sending.
Reconciler	Four passes. MT5 is truth; the database mirrors it and is corrected, never the reverse.
Versioning	Every decision references a strategy version derived from the prompt hash. Editing the prompt registers a new version rather than polluting the old one's trade population.

Side by side

DIMENSION	ORIGINAL	PHASE-5-SIZING-EXITS
Persistence	`memory.json, new_rules.json`	`SQLite, 13 tables, WAL`
Risk authority	Confidence threshold + 2 loop guards	14-check engine with sole authority
Risk per trade	1.0% of equity	0.25% (demo) / 0.5% (prop), profile-driven
Universe	9 symbols	4 symbols
Adaptation	LLM auditor rewrites rules autonomously	None. Strategy is frozen and versioned
Daily loss limit	None	3% with 0.5% buffer, ledger reconciled to MT5
Account floor	None	$90,000 + $500 buffer

Crash safety	Re-reads memory.json on boot	Idempotent comment tags; ambiguity resolved from deal history
Exits	SL/TP + reversal close	SL, TP, REVERSAL, FLATTEN, KILL, MANUAL — all labelled
Broker facts	Read live per order	Measured at startup into profile tables; never assumed
Tests	None	255, against a fake MT5 shim
24/7 operation	WMI detached process, survives logout	`start_bot.bat`, foreground

The one real conflict

> **The self-learning auditor and the Phase 6 calibration gate cannot coexist.** Phase 6 asks whether a confidence score of 80 actually wins more often than a score of 60, and needs ≥200 closed trades under a *frozen* strategy to answer it. The auditor changes what the score means every time it runs — a score of 80 in week one and a score of 80 in week four have been produced by different rule sets. You cannot calibrate a measure that moves while you are measuring it.

This is not an argument against the auditor. It is an argument about *ordering*. The defensible sequence is: freeze the strategy, run Phase 6, find out whether the signal predicts anything at all — and only then turn on a mechanism that adapts it. Adaptive machinery layered on top of an edge that was never demonstrated adapts noise.

If both are wanted eventually

- Store rules in SQLite with a version id, not in a flat JSON file.
- Treat every audit as registering a **new strategy version**, so trade populations never mix — this is invariant 9 on the phase-5 branch.
- Keep the 15–30 clamp and the 2-sample minimum. They are the only things standing between the model and a self-inflicted permanent ban on a symbol.
- Run the auditor only after Phase 6 has returned a verdict.

Which branch to run

IF YOUR GOAL IS…	RUN
Get something autonomous trading a demo account this week, across a wide universe, that visibly improves itself	`original`
Find out whether the AI signal has any predictive value before risking real money	`phase-5-sizing-exits`

Pass a prop-firm evaluation with hard daily-loss and floor rules	`phase-5-sizing-exits`
Explore whether loss-pattern learning works at all	`original`

> **Neither branch has traded a live account.** Everything on `phase-5-sizing-exits` was verified against a fake MT5 shim; everything on `original` has been syntax- and logic-checked but never connected to a broker, because MetaTrader5 is Windows-only and this work was done on macOS. Both need a demo terminal before they need anything else.
